

Notas de aula de COMP0512, Programação C - Programação em C

Daniel Oliveira Dantas

18 de setembro de 2026

Sumário

1	Introdução ao C	1
1.1	Palavras reservadas	1
1.2	Um programa em C mínimo	1
1.3	Programando em C	2
1.4	Literais	3
1.5	Chamada de funções	4
1.6	A biblioteca de funções matemáticas	5
1.7	Comentários	5
1.8	Identificadores	6
1.9	Variáveis	6
1.10	Constantes	7
1.11	Tipos de dados	8
1.12	Expressões numéricas	9
1.12.1	O operador de atribuição =	10
1.12.2	Operadores aritméticos	10
1.12.3	Operadores lógicos e de comparação	11
1.12.4	Operadores bit a bit	12
1.13	<i>Type cast</i>	13
1.14	Diretivas de pré-processamento	13
1.15	Exercícios	14
2	Controle de fluxo	19
2.1	if / else	19
2.2	switch / case	20
2.3	for	21
2.4	while	21
2.5	Exercícios	22
3	Indentação	27
4	Entrada e saída	29
4.1	printf	29
4.2	scanf	31
4.3	sscanf	32

4.4	fscanf	33
4.5	Exercícios	35
5	Tipos de dados compostos	37
5.1	Estruturas compostas heterogêneas	37
5.2	Estruturas compostas homogêneas	40
5.3	Exercícios	41
6	Funções	45
6.1	Passagem de parâmetros por valor	47
6.2	Passagem de parâmetros por referência	47
6.3	Exercícios	48
7	Exemplos de programas	51
7.1	pause.c	51
7.2	ascii.c	51
7.3	dado.c	52
7.4	primos.c	53
7.5	series.c	54
7.6	generate.c	55
7.7	ordena.c	56
A	Tabela ASCII	63

Capítulo 1

Introdução ao C

O C é uma linguagem *case sensitive*, compilada, com tipagem estática e fraca. *Case sensitive* porque letras maiúsculas são consideradas diferentes de minúsculas. Seu código não roda interativamente como nas linguagens interpretadas. Deve ser compilado, ou seja, gerar código executável antes de rodar. Se diz que o C tem tipagem estática pois o tipo de uma variável, uma vez definido, não muda mais. Se diz que C tem tipagem fraca pois é possível realizar operações entre variáveis de tipos diferentes, como somar inteiros e reais.

1.1 Palavras reservadas

- Palavras reservadas ou palavras-chave: são palavras que não podem ser usadas para nomear variáveis ou funções criadas pelo usuário.

<code>char</code>	<code>void</code>	<code>for</code>
<code>double</code>	<code>const</code>	<code>if</code>
<code>float</code>	<code>switch</code>	<code>else</code>
<code>int</code>	<code>case</code>	<code>typedef</code>
<code>long</code>	<code>default</code>	<code>struct</code>
<code>short</code>	<code>break</code>	<code>return</code>
<code>signed</code>	<code>continue</code>	<code>main</code>
<code>unsigned</code>	<code>while</code>	<code>sizeof</code>
<code>static</code>	<code>do</code>	<code>enum</code>

- A lista de palavras reservadas depende da versão do C. As palavras abaixo, por exemplo, surgiram no padrão C23.

```
constexpr bool true false
```

1.2 Um programa em C mínimo

- Um programa em C mínimo: este programa não faz nada, simplesmente declara uma função `main` vazia. A função `main` é o ponto de entrada de

qualquer projeto em C, ou seja, por onde o programa começa a execução. Neste exemplo não há nenhuma instrução dentro da `main`.

```
int main(void)
{
}
```

- Outro programa em C: este programa simplesmente imprime uma mensagem na tela através de uma chamada à função `printf`. Para usar `printf`, é necessário incluir o arquivo de cabeçalho `stdio.h`.

```
#include <stdio.h>
int main(void)
{
    printf("Hello world!\n");
}
```

- Ainda outro programa em C: como no exemplo anterior, imprime uma mensagem na tela. Além disso, espera o usuário pressionar Enter antes de terminar a execução através de uma chamada à função `getchar`.

```
#include <stdio.h>
int main(void)
{
    printf("Hello world!\n");
    printf("Pressione Enter para continuar...\n");
    getchar();
}
```

1.3 Programando em C

Para programar em C é necessário um compilador. Se seu sistema operacional for Windows, instale o Orwell Dev-C++, conhecido como *devcpp*, disponível em <http://sourceforge.net/projects/orwelldevcpp/>. Se seu sistema operacional for Linux, instale o gcc rodando o comando `sudo apt install gcc`

- Como criar um programa em C
 - 1 - Editar o código fonte e salvar em um arquivo com extensão `c`.
 - 2 - Compilar o código fonte usando um compilador.
 - 3 - Rodar o programa.

- Usando o devcpp: o devcpp é um ambiente integrado de programação, com editor e compilador na mesma aplicação.

- 1 - Criar um novo *source file* com ctrl + n.
- 2 - Editar e salvar o arquivo com extensão c.
- 3 - Pressionar F9 para compilar e rodar.
- 4 - Ler as mensagens do compilador e corrigir os erros.
- 5 - *Go to* 3.

- Usando o gcc: o gcc é um compilador de linha de comando.

- 1 - Criar um novo *source file* com um editor de texto como o emacs.
- 2 - Editar e salvar o arquivo com extensão c.
- 3 - Compilar usando o gcc. Antes de rodar a linha de comando abaixo, substitua o conteúdo destacado pelos nomes dos arquivos.

```
gcc nome_arq_fonte.c -o nome_arq_executável
```

- 4 - Ler as mensagens do compilador e corrigir os erros de compilação.
- 5 - Chamar o executável caso tenha compilado e corrigir os erros de lógica.
- 6 - *Go to* 3.

- A linguagem C mudou consideravelmente ao longo dos anos, e, por isso, códigos antigos podem resultar em erros e *warnings* quando usamos novas versões do gcc. O Ubuntu 24 usa o gcc 13, que usa `-std=c17` como versão da linguagem. Já o Ubuntu 26 usa o gcc 15, que usa `-std=c23`. Para usar o padrão C23 com o gcc 13, é necessário usar a opção `std=c2x` na linha de comando.

```
gcc -std=c2x nome_arq_fonte.c -o nome_arq_executável
```

Para descobrir em tempo de execução a versão usada na compilação, pode-se imprimir o valor da macro `__STDC_VERSION__` como na linha de código a seguir.

```
printf("Standard = %ld\n", __STDC_VERSION__);
```

1.4 Literais

Literais são valores com os quais podemos trabalhar. Podem ser de diversos tipos:

Números inteiros:	0 -1 333	Números reais:	2.0 3.1415 -1000.0
Inteiros hexadecimais:	0xff 0x002a 0xd	Inteiros octais:	077 06 02342
Caracteres:	'a' '4' '\n'	Strings:	"Hello world" "bom dia" "Idade = %d anos"
Valores booleanos:	true false		

- Para usar os valores booleanos, é necessário incluir o arquivo de cabeçalho `stdbool.h`. No padrão C23 e no C++, esses valores se tornaram palavras-chave e não é necessário incluir o cabeçalho.

1.5 Chamada de funções

- Funções são trechos de código encapsulados que executam alguma operação.
- Funções são chamadas pelo nome.
- Recebem zero ou mais parâmetros entre parênteses. O número de parâmetros é definido na sua criação e pode ser constante (as funções `sin`, `cos`, `log` recebem um parâmetro; a função `pow` recebe dois parâmetros) ou variável (`printf` recebe um ou mais parâmetros).
- Podem retornar um valor de algum tipo de dado. Esse tipo (real, inteiro, caracter etc) é definido no momento da sua criação. O valor retornado pode mudar dependendo dos parâmetros que a função recebe ao ser chamada.
- Podem também não retornar nada. São as funções de tipo `void`.
- Exemplos de chamadas:

```
printf("Hello\n");
printf("O mundo termina no ano de %d\n", 2012);
sin(1.0);
pow(2.0, 3.0);
printf("O seno de pi/2 eh %f\n", sin(3.14/2.0));
```


1.6 A biblioteca de funções matemáticas

Para usá-la, adicione a linha seguinte ao início do código:

```
#include <math.h>
```

- Caso esteja usando Linux, adicionar a opção `-lm` à linha de comando de chamada ao gcc:

```
gcc -lm nome_arq_fonte.c -o nome_arq_executável
```

- Contém funções como

<code>sin</code>	<code>cos</code>	<code>tan</code>
<code>asin</code>	<code>acos</code>	<code>atan</code>
<code>log</code>	<code>log10</code>	<code>pow</code>
<code>ceil</code>	<code>floor</code>	<code>fabs</code>

- Contém constantes como

<code>M_PI</code>	<code>M_E</code>
-------------------	------------------

- Abaixo está um exemplo de programa que usa essa biblioteca. O programa imprime uma mensagem com o valor do seno de $\pi/4$ através de uma chamada a `printf`.

```
#include <stdio.h>
#include <math.h>
int main(void)
{
    printf("O seno de pi/4 eh %f\n", sin(3.14/4.0));
}
```

1.7 Comentários

Comentários são textos ignorados pelo compilador. Dentro de um comentário é possível escrever qualquer coisa, sem risco de erros de compilação. Em C existem dois tipos de comentário:

- Comentários de uma linha: começa com `//` e vai até o final da linha.
- Comentários de múltiplas linhas: começa com `/*` e termina com `*/`.
- Exemplo de programa com comentários.

```

/*
Programa que imprime o valor do seno
de 45 graus, ou pi/4.
*/
#include <stdio.h> // biblioteca do printf
#include <math.h>  // biblioteca do sin
int main(void)
{
    printf("O seno de pi/4 eh %f\n", sin(3.14/4.0));
}

```

1.8 Identificadores

Identificadores são os nomes de variáveis, funções, constantes e outros objetos, tanto os criados pelo usuário quanto os contidos em bibliotecas.

- O primeiro caracter pode ser letra ou sublinhado
- Os demais caracteres podem ser letras, dígitos ou sublinhado.
- Não pode ser igual a nenhuma palavra reservada.
- Exemplos:

Corretos:	<u>_</u> ABC	Incorretos:	123
	contador		i+1
	i		a!
	x2		r\$

1.9 Variáveis

Variáveis são posições de memória com um nome, usadas para armazenar valores. Os valores armazenados podem ser modificados a qualquer momento.

- O nome de uma variável deve ser um identificador válido, como descrito na Seção 1.8.
- Sua declaração e inicialização devem aparecer no código antes de seu primeiro uso.
- A declaração de uma variável é onde definimos seu tipo e seu identificador, isto é, seu nome.
- A inicialização de uma variável é onde atribuímos a ela algum valor.
- A declaração de uma variável tem o seguinte formato:

tipo de dados identificador;

- Exemplos de declarações de variáveis:

```
int x;  
int i;  
float f;  
char c;  
unsigned int file_size;  
unsigned char idade;  
unsigned short int ano;  
double p1;  
long long int n;  
long double value;
```

- Variáveis podem ser

Locais, quando declaradas dentro de uma função.

Globais, quando declaradas fora de funções.

Parâmetros, quando declaradas na lista de parâmetros da definição de uma função.

- Variáveis locais e globais podem ser inicializadas, ou seja, receber algum valor, no momento de sua declaração. O formato de uma declaração com inicialização é o seguinte:

tipo de dados identificador = valor;

Lembrando que identificador é o nome. O valor pode ser um literal, como descrito na Seção 1.4, outra variável, uma chamada de função que retorna algum valor, ou mesmo uma expressão matemática.

- Exemplos de declarações de variáveis com inicialização:

```
int x = 1;  
int i = 0;  
float e = 2.7;  
char c = 'a';  
double p = sin(M_PI/4.0);  
double p2 = p;  
double p_dup = 2.0 * p;
```

1.10 Constantes

Como variáveis, constantes são posições de memória que armazenam algum valor. Porém ao contrário de variáveis, o valor não pode ser modificado.

- A declaração de uma constante tem o seguinte formato:

```
const tipo de dados identificador = valor;
```

- Exemplos de declarações de constantes:

```
const long int c = 299792458;  
const float pi = 3.141592653589793238462;  
const float G = 6.67428E-11;  
const double h = 6.62606896E-34;
```

- Alternativamente, há a palavra reservada `constexpr`, que garante que o valor é definido em tempo de compilação. Isso é útil, por exemplo, para definir valores usados como um `case` dentro do `switch`. Esta palavra está disponível a partir do padrão C23 e também no C++.

```
constexpr tipo de dados identificador = valor;
```

1.11 Tipos de dados

Em um computador digital, todo e qualquer dado é armazenado em forma de bits. O conteúdo de uma posição de memória pode ser interpretado como um número inteiro, real ou até mesmo uma string. Portanto, para que um dado seja interpretado corretamente, é necessário associar a esse dado o seu tipo. Na linguagem C os tipos mais importantes são

- **int**: valor inteiro. Normalmente ocupa 4 bytes, sendo capaz de representar inteiros de -2147483648 a 2147483647.
- **float**: valor real. Normalmente ocupa 4 bytes, sendo capaz de representar valores reais com cerca de 8 dígitos significativos.
- **double**: valor real. Normalmente ocupa 8 bytes, sendo capaz de representar valores reais com cerca de 16 dígitos significativos.
- **char**: valor inteiro pequeno ou caracter. Ocupa 1 byte, sendo capaz de representar inteiros de -128 a 127.
- **bool**: valor lógico. Pode ser `true` ou `false`. Para usar o tipo `bool`, é necessário incluir o arquivo de cabeçalho `stdbool.h`. No padrão C23 e no C++, esse tipo se tornou palavra-chave e não é necessário incluir o cabeçalho.
- **void**: tipo vazio. Usado para declarar funções que não retornam valor ou ponteiros sem tipo associado.

Alguns tipos de dados podem ser alterados por modificadores:

- **unsigned**: pode modificar os tipos `int` e `char`. Uma variável **unsigned** é sempre positiva, e pode representar valores duas vezes maiores que sua correspondente **signed**
- **signed**: pode modificar os tipos `int` e `char`. Por padrão toda variável `int` e `char` é **signed**, ou seja, com sinal.
- **short**: pode modificar o tipo `int`, diminuindo sua capacidade. Normalmente, em máquinas de 64 bits, um short int ocupa 2 bytes.
- **long**: pode modificar os tipos `int` e `double`, aumentando sua capacidade. Normalmente, em máquinas de 64 bits, um long int ocupa 8 bytes e um long double, 16 bytes.

Para descobrir quanto espaço ocupa uma variável de um certo tipo de dados, usar o operador unário `sizeof`. `sizeof` pode ser usado com tipos de dados ou com variáveis. No primeiro caso, o uso de parênteses é obrigatório, e no segundo, é opcional. Observe o exemplo abaixo.

```
#include <stdio.h>
int main(void)
{
    char c;
    printf("Um int ocupa %ld bytes\n", sizeof(int));
    printf("A variavel c ocupa %ld bytes\n", sizeof(c));
}
```

1.12 Expressões numéricas

Expressões numéricas são compostas por operadores e operandos. Os operandos podem ser literais, funções que retornam algum valor, variáveis, constantes, ou mesmo outras expressões. Os operadores podem ser de atribuição, de comparação, aritméticos, lógicos ou bit a bit.

- Uma expressão elementar contém apenas um operando:

literal

função que retorna valor

variável

constante

- Exemplos de expressões elementares:

```
10
sin(M_PI)
x
```

```
M_PI
```

- Uma expressão composta pode conter vários operadores e operandos:

(expressão operador binário expressão)

(operador unário expressão)

- Exemplos de expressões compostas:

```
2 + 2
-1
x = 10
sin(M_PI) + 1.0
y = (3 + x)
n = 2 * (3 + i)
```

1.12.1 O operador de atribuição =

É usado para atribuir um valor a uma variável. É importante salientar que, antes de usar uma variável, ela deve ser inicializada, ou seja, algum valor deve ser atribuído a essa variável. Uma variável não inicializada pode conter um valor imprevisível.

- A forma geral de uma atribuição é a seguinte:

identificador da variável = expressão;

Onde é claro que **expressão** pode ser elementar, como um valor literal, outra variável, uma constante, uma chamada a função; ou composta, como uma operação aritmética, lógica etc.

- Exemplos de atribuições:

```
n = 10;
m = n;
y = - (3 + x);
n = 3 * (4 + i);
```

1.12.2 Operadores aritméticos

Expressões com operadores aritméticos retornam valores numéricos, inteiros ou reais.

Precedência:			
maior	++	--	(incremento e decremento unário)
	-		(menos unário)
	*	/	% (multiplicação, divisão e módulo)
menor	+	-	(soma e subtração)

- Operadores de maior precedência são avaliados antes dos de menor precedência.
- Operadores com mesmo nível de precedência são avaliados da esquerda para a direita.
- É possível usar parênteses para modificar a ordem das operações
- `i++` é o mesmo que `i = i + 1`.
- `i--` é o mesmo que `i = i - 1`.
- Nos exemplos abaixo, que valor é atribuído à variável à esquerda do operador =?

```
a = 2 * 3 + 1;
b = 1 + 2 * 3;
c = 10 / 2 * 5;
d = 10 % 3;
e = 2 * 10 % 4;
f = 3 * (2 + 1);
g = 2;
h = 3 * ++g;
i = 3 * g++;
```

1.12.3 Operadores lógicos e de comparação

Expressões com operadores lógicos ou de comparação retornam valores do tipo `bool`, ou seja, `true` ou `false`.

Precedência:					
maior	!				(negação lógica)
	>	>=	<	<=	(desigualdade)
	==	!=			(igualdade e diferença)
	&&				(and lógico)
menor					(or lógico)

- Nos exemplos abaixo, que valor é atribuído à variável à esquerda do operador =?

```

a = 1 < 2;
b = true || false;
c = true && false;
d = 1 < 2 || 4 < 2;
e = 1 < 2 && 4 < 2;
f = 1 < 1;
g = !(3 < 2);
e = 3 == 4;
e = 3 != 4;

```

- Para testar se o valor de uma variável está em um certo intervalo, a construção é a seguinte:

```
f > 0 && f < 1
```

para testar se f está entre 0 e 1. Jamais omitir o **&&** como na expressão INCORRETA abaixo:

```
0 < f < 1
```

1.12.4 Operadores bit a bit

Operam sobre o valor binário de uma variável. A variável só pode ser do tipo **int**, **char** ou suas variantes.

Precedência:		
maior	~	(complemento de um)
	<< >>	(shift left e shift right)
	&	(and binário)
	^	(xor binário)
menor		(or binário)

- Nos exemplos abaixo, que valor é atribuído à variável à esquerda do operador =?

```

a = 3 | 1;
b = 1 >> 1;
c = 1 << 1;
d = 1 << 2;
e = 5 ^ 3;
f = ~ 0;

```


1.13 *Type cast*

A linguagem C usa os tipos dos operandos para definir o tipo do resultado. Por isso, na operação abaixo, o resultado obtido é zero, mesmo sendo a variável `result` de tipo `float`.

```
int num = 1;
int den = 3;
float result = num / den;
printf("float = %f\n", result);
```

Para evitar isso, podemos fazer um *type cast* dos operandos para `float`, forçando o resultado a ser também de tipo `float`. A sintaxe do *type cast* é colocar o tipo entre parênteses antes da variável a ser modificada como no exemplo abaixo.

```
float result = (float)num / (float)den;
```

1.14 Diretivas de pré-processamento

Diretivas de pré-processamento são usadas pelo compilador para modificar o código-fonte antes de iniciar a compilação propriamente dita. São indicadas pelo símbolo `#`.

- Um exemplo já visto anteriormente é o `#include`. A linha abaixo, antes da compilação, é substituída pelo conteúdo do arquivo de cabeçalho `stdio.h`

```
#include <stdio.h>
```

- Outro exemplo é o `#define`, usado para definir macros. Macros são identificadores que são substituídos pela string que o sucede ao longo do código-fonte. No exemplo abaixo, a primeira linha define `TAM_MAX` como 10, portanto, antes da compilação, esse identificador será substituído por 10 na declaração de `array`.

```
#define identificador string
```

```
#define TAM_MAX 10
int array[TAM_MAX]
```

- O identificador pode ser usado para definir macros com parâmetros, como no exemplo a seguir. Deve-se ter cuidado pois, como a substituição é puramente textual, o valor de `x` pode não ser avaliado corretamente a depender da precedência dos operadores. Isso pode ser evitado com os parênteses ao redor `x`, que garantem que seu valor é calculado antes da multiplicação.

```
#define QUADRADO(x) ((x)*(x))
printf("%d\n", QUADRADO(5+5));
```

1.15 Exercícios

1. Crie uma função `main` vazia.
2. Crie uma função `main` e dentro dela declare as seguintes variáveis:
 - (a) um inteiro chamado `n`.
 - (b) um real de precisão simples chamado `f`.
 - (c) um real de precisão dupla chamado `d`.
 - (d) um caracter chamado `c`.
 - (e) um inteiro longo chamado `l`.
3. Em C, as instruções devem ser terminadas com ponto e vírgula. A declaração de uma variável é o primeiro lugar do programa onde a variável aparece e é onde seu tipo de dados é definido. A inicialização é onde algum valor é atribuído a ela pela primeira vez. A declaração e inicialização podem ocorrer na mesma instrução. Por exemplo:

```
int i = 0;
float x = 5.2;
float y = x*x;
```

Crie uma função `main` e dentro dela declare e inicialize na mesma instrução:

- (a) um inteiro chamado `val`.
- (b) um real de precisão simples chamado `x`.
- (c) um real de precisão dupla chamado `xd`.
- (d) um caracter chamado `tiny`.
- (e) um inteiro longo chamado `huge`.

4. Crie uma função `main` que faz o mesmo que na questão anterior e em seguida, para cada variável, imprime uma mensagem com seu nome, a palavra "vale", e seu valor. Para uma variável chamada pipoca, por exemplo:

```
pipoca vale 3.141592
```

5. Crie uma função `main` que faz o mesmo que na questão anterior, porém, ao invés de declarar variáveis, declare constantes.

6. Crie uma função `main` e dentro dela declare e inicialize na mesma instrução cinco variáveis inteiras. Para inicializá-las use expressões com três operadores aritméticos cada. Na mesma linha, coloque em um comentário o resultado da expressão. Por exemplo:

```
int i = 5 + 2 * 3 * 4; // 29
```

7. Crie uma função `main` e dentro dela declare e inicialize na mesma instrução cinco variáveis inteiras. Para inicializá-las use expressões com cinco operadores aritméticos cada. Na mesma linha, coloque em um comentário o resultado da expressão.

8. Crie uma função `main` e dentro dela declare e inicialize na mesma instrução cinco variáveis reais de precisão dupla. Para inicializá-las use expressões com dois operadores aritméticos e uma chamada a função em cada expressão. Na mesma linha, coloque em um comentário o resultado da expressão.

9. Crie uma função `main` e dentro dela declare e inicialize na mesma instrução cinco variáveis reais de precisão simples. Para inicializá-las use expressões com três operadores aritméticos cada. Na mesma linha, coloque em um comentário o resultado da expressão.

10. Crie uma função `main` que imprime a mensagem "Hello\n".

11. Crie uma função `main` que imprime a mensagem "Hello\n", "Pressione Enter para continuar\n", e em seguida chama `getchar()`.

12. O que acontece quando:

(a) atribuímos um valor `int` a uma variável `float`?

(b) atribuímos um valor `float` a uma variável `int`?

13. Crie um programa que calcule e imprima a área das figuras planas especificadas. Para um paralelogramo com lados 10 e 4, com ângulo de 60 graus entre eles, por exemplo:

```
float a = 10;
float b = 4;
float deg = 60;
float h = b * sin(deg * M_PI / 180);
float area = a * h;
printf("area = %f\n", area);
```

- (a) um retângulo com lados 3 e 6.
 - (b) um quadrado com lado 5.
 - (c) um triângulo retângulo com catetos 3 e 4.
 - (d) um triângulo retângulo com cateto igual a 5 e hipotenusa igual a 13.
 - (e) um triângulo equilátero com lado 4.
 - (f) um círculo com raio 2.
14. Crie um programa que calcule e imprima o volume dos sólidos especificados.
- (a) um cubo com aresta igual a 5.
 - (b) um paralelepípedo com lados 3, 4 e 5.
 - (c) um cilindro cuja base tem raio igual a 3 e altura igual a 4.
 - (d) uma pirâmide com base quadrada de lado 5 e altura igual a 6.
 - (e) uma esfera com raio igual a 1.
 - (f) um prisma cuja base é um triângulo equilátero com lado igual a 2 e altura igual a 10.
15. Considere um material com densidade de 0.420 g/cm^3 . Crie um programa que calcule e imprima a massa em g e kg dos objetos descritos abaixo.
- (a) uma folha de $19 \times 270 \text{ cm}$ e espessura de 0.6 mm .
 - (b) uma chapa de $210 \times 82 \text{ cm}$ e espessura de 3.5 cm .
 - (c) uma ripa de $1 \times 2 \text{ cm}$ e comprimento de 1.2 m .
 - (d) um sarrafo de $3.5 \times 4 \text{ cm}$ e comprimento de 3 m .
 - (e) um caibro de $5 \times 5 \text{ cm}$ e comprimento de 3 m .
 - (f) uma viga de $4 \times 9 \text{ cm}$ e comprimento de 4 m .
 - (g) um tarugo com diâmetro de 15 mm e comprimento de 60 cm .
16. Considere os vetores $\mathbf{v} = (2, 2, 0)$ e $\mathbf{w} = (3, -3, 0)$. Crie um programa que calcule e imprima
- (a) o vetor $\mathbf{v} + \mathbf{w}$.
 - (b) o vetor $\mathbf{v} - \mathbf{w}$.

- (c) a norma de $\mathbf{v}$.
 - (d) a norma de $\mathbf{w}$.
 - (e) o coeficiente angular de $\mathbf{v}$, i.e. o ângulo formado entre o vetor e o eixo x .
 - (f) o coeficiente angular de $\mathbf{w}$.
 - (g) o produto escalar de $\mathbf{v}$ por $\mathbf{w}$.
 - (h) o produto vetorial de $\mathbf{v}$ por $\mathbf{w}$.
17. Considere a equação de segundo grau $x^2 - 3x - 10 = y$. Crie um programa que calcule e imprima
- (a) o valor de $\Delta = b^2 - 4ac$.
 - (b) suas duas raízes reais, dadas por $x = (-b \pm \sqrt{\Delta})/2a$.
 - (c) a coordenada x do vértice, dada por $x = -b/2a$.
 - (d) a coordenada y do vértice, dada por $y = -\Delta/4a$.
18. Sabendo que $\log_a b = \log b / \log a$, crie um programa que calcule e imprima
- (a) o valor de $\log_2 1024$.
 - (b) o valor de $\log_3 729$.
 - (c) o valor de $\log_4 256$.
 - (d) o valor de $\log_5 625$.
19. Crie um programa que imprima em uma tabela os valores de 0 a 100 graus Celsius em intervalos de 10 graus na primeira coluna. Na segunda coluna imprima os valores correspondentes em Kelvin e na terceira, os valores correspondentes em Fahrenheit. Seja c o valor em Celsius, k o valor em Kelvin e f o valor em Fahrenheit, considere que $k = c + 273.15$ e $f = c \times 9/5 + 32$.

Capítulo 2

Controle de fluxo

Um programa de computador é uma sequência de instruções. Normalmente as instruções são executadas na ordem em que aparecem no programa. Porém, isso permite apenas a criação de programas muito simples.

Imagine que tenhamos por exemplo uma matriz com centenas de valores. Para processar cada item dessa matriz, precisamos executar centenas de instruções. Se a instrução for a mesma para todos os itens da matriz, podemos facilitar nossa vida criando um programa bem mais simples se colocarmos essa instrução dentro de um *laço*, ao invés de ter uma cópia desta mesma instrução para cada item da matriz.

Outro caso em que o controle de fluxo é útil é quando uma instrução é executada em apenas alguns casos. Imagine que temos um banco de dados com milhões de valores de CPF e queremos imprimir apenas o nome do portador de um certo CPF. Podemos usar uma instrução do tipo `if` que, para cada CPF lido, testa se é igual ao CPF desejado e imprime o nome do portador somente em caso afirmativo.

2.1 `if / else`

Uma instrução do tipo `if` executa um bloco de instruções somente se uma dada condição é verdadeira. Opcionalmente pode ser complementada com uma instrução `else`, que executa outro bloco caso a dada condição seja falsa.

- A forma geral de uma instrução `if` é a seguinte:

```
if (expressão condicional)
{
    bloco executado se a condição for verdadeira
}
```

- A forma geral de uma instrução `if/else` é a seguinte:

```

if (expressão condicional)
{
    bloco executado se a condição for verdadeira
}
else
{
    bloco executado se a condição for falsa
}

```

- Observe que a expressão condicional só aparece uma vez, junto ao `if`, jamais junto ao `else`.
- A expressão condicional pode ser uma expressão lógica, de comparação, ou uma operação aritmética com inteiros. Não usar expressões aritméticas com `float` ou `double`.
- O bloco do `if` é executado se a expressão condicional der como resultado `true` ou diferente de zero.
- O bloco do `else` é executado se a expressão condicional der como resultado `false` ou igual a zero.

2.2 switch / case

Uma instrução do tipo `switch` avalia uma expressão inteira de tipo `int`, `char`, `short` ou `long`. Pode selecionar, dentre vários casos, aquele correspondente ao valor da expressão. O bloco de código correspondente ao caso inicia com a palavra-chave `case` seguida de um valor literal e dois pontos.

- A forma geral de uma instrução `switch` é a seguinte:

```

switch (expressão inteira)
{
    case valor inteiro:
        bloco executado se a expressão for igual ao valor
        break;
    case valor inteiro:
        bloco executado se a expressão for igual ao valor
        break;
    ...
    default:
        bloco executado caso contrário
        break;
}

```


- O bloco de cada caso deve terminar em **break** para que a execução salte para o final do **switch**. Sem o **break**, a execução continua para o bloco de instruções do **case** seguinte.
- O caso **default** é executado se nenhum dos valores casar com a expressão. Por convenção vem sempre ao final do bloco do **switch**.
- A expressão usada no **switch** pode ser uma variável. No **case**, porém, variáveis não são permitidas. Em C, a partir do padrão C23, é permitido o uso de constantes declaradas com **constexpr**. Em C++ é permitido o uso de constantes.

2.3 for

Palavra reservada para criar instruções de iteração, ou seja, laços. Um laço é um bloco que é executado várias vezes.

- A forma geral de uma instrução **for** é a seguinte:

```
for (inicialização; condição de continuidade; incremento)
{
    bloco a ser executado várias vezes
}
```
- Na inicialização, geralmente uma variável de controle é declarada e inicializada.
- Variável de controle é a que indica quando o laço deve parar.
- Na condição de continuidade, se testa se o laço deve continuar. Geralmente isso é feito comparando a variável de controle com algum valor.
- No incremento, a variável de controle é incrementada. Pode também ser decrementada se necessário.

2.4 while

Faz exatamente o mesmo que o **for**, ou seja, serve para criar laços. A diferença está na sintaxe.

- A forma geral de uma instrução **while** é a seguinte:

```
inicialização;
while (condição de continuidade)
{
    bloco a ser executado várias vezes
    incremento;
}
```

- A inicialização, condição de continuidade e incremento funcionam exatamente da mesma maneira que no `for`, conforme explicado na Seção 2.3
- Às vezes é necessário executar o bloco uma vez antes de testar a condição de continuidade. Nesse caso, usamos a forma alternativa abaixo com `do`:

```
inicialização;
do
{
    bloco a ser executado várias vezes
    incremento;
}
while (condição de continuidade);
```

2.5 Exercícios

1. Crie uma função `main` que imprime todos os inteiros de 1 a 100 com apenas uma instrução com `printf`. Use:
 - (a) um laço com a palavra reservada `while`.
 - (b) um laço com a palavra reservada `for`.
2. Seja `n` um inteiro entre 1 e 1000. Crie uma função `main` com um laço (`for` ou `while`) que imprime uma mensagem "`%d modulo 2 vale %d`" para cada `n` entre 1 e 1000. No lugar do primeiro `%d` deve aparecer o valor de `n`, e no lugar do segundo `%d`, o valor de `n` modulo 2. O operador módulo, representado por `%`, retorna o resto da divisão de um número por outro número. Por exemplo, `12 % 10` retorna 2.

Resposta:

```
int main(void)
{
    for(int i = 1; i <= 1000; i++)
    {
        printf("\nd modulo 2 vale %d\n", i, i % 2);
    }
}
```

3. Seja `n` um inteiro entre 1 e 1000. Crie uma função `main` com um laço (`for` ou `while`) que imprime uma mensagem "`%d eh par`" se `n` for par, ou "`%d eh impar`" se `n` for impar, para cada `n` entre 1 e 1000. No lugar do primeiro `%d` deve aparecer o valor de `n`. Usar `if` para testar se o módulo de `n` por 2 vale 0, o que indica que `n` é par, ou 1, o que indica que `n` é ímpar. Observe que operador de comparação que testa igualdade em C é o `==`, diferente do operador de atribuição `=`.

4. Seja $v = \sum_{i=0}^{10} i$. Crie uma função `main` que calcula v usando:
- (a) um laço com a palavra reservada `while`.
 - (b) um laço com a palavra reservada `for`.
5. Seja n um número inteiro. Crie uma função `main` que calcula seu fatorial usando um laço. Qual o maior n de tipo `int` que o C é capaz de calcular? E de tipo `long int`?
6. Crie um programa que calcula com um laço o produtório abaixo para $n = 10$.
- (a)

$$\prod_{i=1}^n i = n!$$

7. Refaça a questão anterior pedindo ao usuário que forneça um valor de n .
8. Crie um programa que calcula as séries abaixo para $n = 10$. Calcule a somatória com um laço. Calcule também o lado direito da equação e imprima ambos os resultados para comparação.
- (a)

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}$$

(b)

$$\sum_{i=1}^n 2i = n^2 + n$$

(c)

$$\sum_{i=1}^n 2i - 1 = n^2$$

(d)

$$\sum_{i=0}^n 2^i = 2^{n+1} - 1$$

(e)

$$\sum_{i=1}^n i^2 = \frac{1}{6}n(n+1)(2n+1)$$

(f)

$$\sum_{i=1}^n i^3 = \left(\frac{n(n+1)}{2} \right)^2$$

(g)

$$\sum_{i=1}^n \frac{1}{i(i+1)} = \frac{n}{n+1}$$

(h)

$$\sum_{i=1}^n 3i - 2 = \frac{n(3n-1)}{2}$$

(i)

$$\sum_{i=1}^n 5 + 2(i-1) = \frac{n(10 + 2(n-1))}{2}$$

(j)

$$\sum_{i=1}^n i(2)^{i-1} = 1 + (n-1)2^n$$

(k)

$$\lim_{n \rightarrow \infty} \sum_{k=0}^n \frac{1}{k!} = e$$

(l)

$$\lim_{n \rightarrow \infty} \sum_{k=0}^n \frac{(-1)^k}{2k+1} = \frac{\pi}{4}$$

(m)

$$\sum_{i=1}^m \sum_{j=1}^n ij = \frac{mn(m+1)(n+1)}{4}$$

9. Refaça a questão anterior pedindo ao usuário que forneça um valor de n .
10. Crie um programa que calcula a soma dos 100 primeiros termos das séries abaixo.

(a)

$$\sum_{i=1}^{\infty} \frac{1}{i}$$

(b)

$$\sum_{i=0}^{\infty} \frac{1}{2^i}$$

11. Crie um programa que calcula as séries da questão anterior enquanto a parcela é maior que 10^{-3} .
12. Uma relação de recorrência é uma equação segundo a qual o n -ésimo termo de uma sequência é dado por alguma combinação dos termos anteriores. O método de Newton é um exemplo de relação de recorrência.

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

O processo é repetido até que o valor de $f(x_n)$ seja suficientemente próximo de zero, ou seja, até que tenha módulo menor que um certo erro ε . Use o método de Newton para encontrar

- (a) duas raízes de $f(x) = x^2 - 9$.
- (b) duas raízes de $f(x) = x^2 + x - 2$.
- (c) uma raiz de $\cos(x)$ entre 0 e 3.
13. Crie um programa que calcula o produto dos 100 primeiros termos dos produtórios abaixo.
- (a)

$$\prod_{k=1}^{\infty} \left(\frac{2k}{2k-1} \frac{2k}{2k+1} \right) = \frac{2}{1} \frac{2}{3} \frac{4}{3} \frac{4}{5} \frac{6}{5} \frac{6}{7} = \frac{\pi}{4}$$

14. O número de combinações de n itens k a k é dado pela equação abaixo.

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

- (a) Peça ao usuário que insira n e k . Use-os para calcular $\binom{n}{k}$.
- (b) Faça dois laços e imprima as dez primeiras linhas do triângulo de Pascal, ou seja, para cada linha n de 0 a 9, imprima os valores de $\binom{n}{k}$ com k de 0 a n .
15. Crie um programa que testa se um certo n é primo. Use um laço para testar se os inteiros menores que n o dividem.

- (a) Teste para $n = 101$.
- (b) Solicite para o usuário que insira algum valor de n .
- (c) Teste para n variando de 2 a 1000 e imprima o total de números primos encontrados.
- (d) Teste para n variando de 3 a 9999 pulando os pares e imprima o total de números primos encontrados.

Capítulo 3

Indentação

Indentação é a maneira como avançamos ou recuamos as instruções no texto do código fonte para facilitar o entendimento. A linguagem C ignora completamente a indentação. Porém, para o programador, enquanto um código mal indentado pode aparecer completamente ilegível, o mesmo código bem indentado costuma aparecer claro e compreensível.

Observe o código abaixo:

```
#include <stdio.h>
int main (void){long int val=1;for(int n=1;
n<=8;n++){val=val*n;}printf("val = %ld\n",val);}
```

Esse código possui exatamente as mesmas instruções que o código a seguir. A única diferença está na indentação. Observe como o código abaixo é mais legível:

```
#include <stdio.h>
int main (void)
{
    long int val = 1;
    for (int n = 1; n <= 8; n++)
    {
        val = val * n;
    }
    printf("val = %ld\n", val);
}
```

Em C, *bloco* se refere a linhas de código entre chaves. Abrimos um novo bloco de código quando declaramos uma função, quando criamos uma instrução de laço com `for` ou `while`, ou uma instrução de controle de fluxo com `if` ou `switch`. Para se indentar um código em C como no exemplo acima, siga as instruções a seguir:

- A chave que abre um bloco fica abaixo da primeira letra da instrução que abre o bloco.
- Cada vez que se abre uma chave, avançar 4 espaços. Cada vez que se fecha uma chave, recuar 4 espaços.
- A chave que fecha um bloco fica exatamente abaixo da chave que abre o bloco.
- Cada vez que se abre uma chave, pular uma linha para que a primeira instrução do bloco fique na linha seguinte à da chave.

Todo código deve ser indentado corretamente, tanto código escrito em papel quanto em arquivos digitais.

Capítulo 4

Entrada e saída

Entrada e saída se refere à comunicação do programa, tanto com o usuário quanto com arquivos em disco. Duas funções que podem ser usadas para fazer entrada e saída com o usuário pelo console são `printf` e `scanf`. Para fazer entrada e saída com arquivos, pode-se usar `fprintf` e `fscanf`. Para usar qualquer uma dessas funções, adicionar a seguinte linha ao início do código:

```
#include <stdio.h>
```

4.1 printf

`printf` não é uma palavra reservada como `while`, e sim uma função. `printf` espera como primeiro parâmetro uma string, como nos exemplos abaixo.

```
printf("Hello world\n");  
printf("bom dia");  
printf("Pressione enter para continuar...\n");
```

Observe que strings ficam entre aspas duplas. Ao final de algumas das strings acima está o caractere especial `'\n'`. Esse caractere representa um pulo para a linha seguinte. Sempre adicionar esse caractere ao final quando a string contiver uma mensagem que deve aparecer isolada em uma linha.

Além do pulo de linha, existem outros caracteres especiais que `printf` reconhece, como pode ser visto na Tabela 4.1

Caractere	Valor ASCII	Representação
Pula linha	10	\n
Tabulação	9	\t
<i>Beep</i>	7	\a
<i>Backslash</i>	92	\\
Aspas duplas	34	\"
Símbolo de percentagem	37	%%

Tabela 4.1: Caracteres especiais do C

Para ser mais versátil, `printf` pode imprimir valores de expressões no meio da mensagem. Para isso é necessário colocar na string um especificador de formato no ponto em que o valor deve aparecer, e adicionar como parâmetro a expressão a ser impressa. Especificadores de formato são precedidos pelo símbolo % e servem para definir como interpretar o valor da expressão, ou seja, se é um valor inteiro, real, caractere etc. Observe o exemplo abaixo.

```
printf("Um inteiro ocupa %d bytes\n", sizeof(int));
printf("PI vale %f\n", 3.141592653589793238462);
```

Cada uma das strings possui um especificador de formato. A primeira possui um %d para imprimir um número inteiro e a segunda possui um %f para imprimir um número real. A saída do exemplo acima será a seguinte:

```
Um inteiro ocupa 4 bytes
PI vale 3.141593
```

Por padrão o %f imprime o número real com seis casas decimais, mas esse comportamento pode ser modificado colocando logo depois do símbolo % um ponto seguido do número de casas decimais desejado, como a seguir.

```
printf("PI vale %.10f\n", 3.141592653589793238462);
```

cuja saída será

```
PI vale 3.1415926536
```

O formato de números inteiros também pode ser modificado. Colocando um número *n* de dígitos entre o % e o d, fará cada inteiro ocupar *n* espaços. Isso é útil para se imprimir tabelas onde o dígito menos significativo deve ficar alinhado à direita. Para completar o inteiro com zeros à esquerda, colocar um zero entre o % e o número de dígitos completará o número impresso com zeros à esquerda até que ocupe o tamanho desejado. Observe os exemplos abaixo e compare suas saídas.

Saída	Especificador	Exemplo
Inteiro decimal	%d	45
Inteiro decimal long int	%ld	12345678901234
Inteiro hexadecimal	%x	f3 ou F3
	%X	f3 ou F3
Número real float	%f	3.141593
Número real double	%lf	3.141593
Número real em notação científica	%e	1.000000e+09
	%E	1.000000E+09
Caractere	%c	a
String	%s	Hello
Endereço de ponteiro	%p	0x7ffec2e33a8

Tabela 4.2: Especificadores de formato do C

```
printf("Um inteiro ocupa %d bytes\n", sizeof(int));
printf("Um inteiro ocupa %6d bytes\n", sizeof(int));
printf("Um inteiro ocupa %06d bytes\n", sizeof(int));
```

terão como saída respectivamente

```
Um inteiro ocupa 4 bytes
Um inteiro ocupa      4 bytes
Um inteiro ocupa 000004 bytes
```

Em uma mesma string pode haver vários especificadores de formato, bastando que, para cada um dos especificadores, deve haver um valor correspondente como parâmetro. Observe o exemplo abaixo. Existem dois especificadores de formato. O primeiro será substituído pelo valor de `val1` e o segundo, pelo valor de `val2`.

```
int val1 = 33;
float val2 = 1.23;
printf("A variavel inteira vale %d e a variavel real vale %f\n",
      val1, val2);
```

4.2 scanf

A chamada a `scanf` é semelhante à chamada ao `printf`. O primeiro parâmetro também é uma string, mas deve possuir um especificador de formato. Os demais parâmetros são os endereços das posições de memória onde os dados serão gravados. É recomendável porém usar uma chamada `scanf` para cada valor a ser lido. Para passar o endereço de memória, é necessário preceder a variável pelo símbolo `%`.

```
int i;
scanf("%d", &i);
float x;
scanf("%f", &x);
char letra;
scanf("%c", &letra);
```

- `scanf` retorna o número de valores lidos com sucesso. Idealmente deve-se usar esse valor para testar se a leitura foi bem sucedida.
- Segue um exemplo de código com `scanf`.

```
#include <stdio.h>
int main (void)
{
    int result;
    float userfloat;
    printf("Digite um numero real:\n");
    result = scanf("%f", &userfloat);
    printf("result = %d, real digitado = %f\n", result,
        userfloat);
}
```

- No exemplo acima, `userfloat` armazena o real digitado pelo usuário. A variável `result` será igual a 1 caso a leitura seja bem sucedida, e 0 caso contrário.

4.3 sscanf

A função `scanf` pode não funcionar corretamente quando o usuário digita valores de tipos diferentes do esperado. Uma maneira de contornar isso é armazenando a string em uma variável intermediária antes de fazer a leitura. Nesse caso, a leitura é feita com `sscanf`, uma versão alternativa de `scanf` que lê dados de strings.

```
#include <stdio.h>
int main (void)
{
    int result;
    float userfloat;
    char line[256];
    printf("Digite um numero real:\n");
    gets(line);
```

```

result = sscanf(line, "%f", &userfloat);
printf("result = %d, real digitado = %f\n", result, userfloat);
}

```

- No exemplo acima, podemos observar uma nova variável chamada `line`, que armazenará o texto digitado pelo usuário. `gets` lê o texto digitado e o armazena em `line`, que é passado como parâmetro para `sscanf`. As demais partes do código permanecem como no exemplo da Seção 4.2. Note que se a linha lida for maior que o espaço alocado de 256 bytes para `line`, outras variáveis ou código binário podem ser sobrescritos. Isso é chamado *buffer overflow*, e pode ser evitado usando `fgets`, que permite limitar o número de bytes lidos.

4.4 fscanf

A função `fscanf` pode ser usada para leitura de dados de arquivos. Recebe um parâmetro a mais que `scanf`: o ponteiro para o arquivo a ser lido.

- Para obter o ponteiro para o arquivo, usar a função `fopen`. No exemplo abaixo, `fp` é a variável que armazenará o ponteiro para o arquivo. O primeiro parâmetro é uma string que contém o nome do arquivo a ser aberto. Neste exemplo é `arquivo.txt` mas poderia ser qualquer outro. O segundo parâmetro indica o tipo de acesso desejado, ou seja, leitura ou escrita. Nesse caso contém uma letra `r` para indicar que o arquivo será aberto para leitura. Usar a letra `w` para escrita. A função `fopen` retorna zero em caso de erro, ou seja, se o arquivo não existir ou se não houver permissão para o acesso desejado.

```
FILE* fp = fopen("arquivo.txt", "r");
```

- Uma maneira de testar se o arquivo foi aberto corretamente é mostrada a seguir. Uma mensagem mais esclarecedora para o usuário, e portanto mais útil, pode incluir o nome do arquivo.

```

if (fp == 0)
{
    printf("Erro ao abrir arquivo\n");
    return 1;
}

```

- Uma vez que o arquivo esteja aberto, chamar **fscanf** com o ponteiro retornado por **fopen** como primeiro parâmetro. Como **scanf**, recebe também uma string com o especificador de formato, e a variável onde o valor lido deve ser armazenado. Essa variável deve ser passada por referência, ou seja, é necessário usar o símbolo **&** antes do identificador da variável para indicar que é passado seu endereço como parâmetro.

```
int i;
fscanf(fp, "%d", &i);
float x;
fscanf(fp, "%f", &x);
char letra;
fscanf(fp, "%c", &letra);
```

- Segue um programa exemplo que lê dois números inteiros de um arquivo. Cada número deve estar em uma linha diferente no arquivo.

```
#include <stdio.h>
int main (void)
{
    int a, b;
    FILE* fp = fopen("arquivo.txt", "r");
    if (fp == 0)
    {
        printf("Erro ao abrir arquivo\n");
        return 1;
    }
    fscanf(fp, "%d", &a);
    printf("Valor de a = %d\n", a);
    fscanf(fp, "%d", &b);
    printf("Valor de b = %d\n", b);
    fclose(fp);
}
```

- Abaixo está uma versão alternativa que usa **fgets** para ler cada linha do arquivo por completo antes de tentar extrair dela o número. Observe que a função **fgets** é usada no lugar da **gets**, da Seção 4.3.

```
#include <stdio.h>
int main (void)
{
    int MAX_LEN = 256
    int a, b;
    FILE* fp = fopen("arquivo.txt", "r");
```

```

if (fp == 0)
{
    printf("Erro ao abrir arquivo\n");
    return 1;
}
char line[MAX_LEN];
fgets(line, MAX_LEN, fp);
sscanf(line, "%d", &a);
printf("Valor de a = %d\n", a);
fgets(line, MAX_LEN, fp);
sscanf(line, "%d", &b);
printf("Valor de b = %d\n", b);
fclose(fp);
}

```

- Idealmente, deve-se testar se cada `fscanf` e `fgets` fez uma leitura bem sucedida, como no trecho de código abaixo. `fgets` recebe três parâmetros: um vetor de caracteres ou string, o tamanho máximo da string e o ponteiro para o arquivo. Retorna zero em caso de erro.

```

char* result1 = fgets(line, MAX_LEN, fp);
int result2 = sscanf(line, "%d", &n);
if (result1 != 0 && result2 == 1)
{
    printf("Valor lido = %d\n", result, n);
}
else
{
    printf("Valor invalido\n");
}

```

- Ao final do programa, após fazer todas as leituras necessárias, fechar o arquivo com `fclose`, como na linha de código a seguir. Isso salva as escritas no arquivo e o libera para que outros programas possam acessá-lo.

```

fclose(fp);

```

4.5 Exercícios

1. Crie uma função `main` e dentro dela:

- (a) imprima a mensagem "Favor digitar um valor inteiro \n".
 - (b) declare uma variável inteira chamada `i`.
 - (c) use `scanf` para ler o inteiro digitado pelo usuário e salvá-lo na variável `i`.
 - (d) imprima a mensagem "Voce digitou " seguida do valor digitado e "\n".
 - (e) chame `getchar()`.
2. Crie uma função `main` e dentro dela:
- (a) imprima a mensagem "Favor digitar um valor inteiro\n".
 - (b) declare as variáveis inteiras `i` e `result`.
 - (c) use `scanf` para ler o inteiro digitado pelo usuário e salvá-lo na variável `i`. Armazene o valor de retorno de `scanf` em `result`.
 - (d) caso o valor de `result` seja igual a 1 (um), imprima a mensagem "Voce digitou " seguida do valor digitado e "\n".
 - (e) caso contrário, imprima a mensagem "Eu pedi um inteiro!\n".
 - (f) chame `getchar()`.
3. Crie uma função `main` e dentro dela:
- (a) solicite ao usuário que forneça dois valores reais e armazene-os em duas variáveis chamadas `a` e `b`.
 - (b) para cada variável lida, se a leitura foi bem sucedida imprima o valor lido. Caso contrário imprima uma mensagem de erro e finalize o programa.
 - (c) imprima mensagens indicando o valor da soma, do produto, da diferença, da razão entre `a` e `b`, norma do vetor (a,b) e o valor de `a` elevado a `b`.
4. Crie uma função `main` e dentro dela:
- (a) solicite ao usuário que forneça três valores inteiros e armazene-os em três variáveis chamadas `a`, `b` e `c`.
 - (b) para cada variável lida, se a leitura foi bem sucedida imprima o valor lido. Caso contrário imprima uma mensagem de erro e finalize o programa.
 - (c) imprima mensagens indicando o valor das raízes da equação de segundo grau igual a $a*x*x + b*x + c$. Usar as funções das questões `baskara` e `delta` criadas anteriormente.

Capítulo 5

Tipos de dados compostos

Nem todos os problemas podem ser resolvidos com variáveis simples como as primitivas já vistas. Bancos de dados requerem o uso de registros com campos diferentes para armazenar por exemplo o nome, endereço, estado civil etc de cada cliente. Problemas que envolvem matrizes, vetores, listas e sequências requerem o uso de estruturas indexadas, ou seja, que permitam acessar componentes individuais através de índices inteiros. Neste capítulo veremos como criar, modificar e consultar estruturas com tipos de dados compostos, que podem ser homogêneas, compostas por itens de mesmo tipo, ou heterogêneas, compostas por itens de tipos diferentes.

5.1 Estruturas compostas heterogêneas

As estruturas compostas heterogêneas são também chamadas de registros. São úteis na criação de bancos de dados e podem armazenar em conjunto dados numéricos e de texto. Abaixo está a sintaxe da criação de um tipo de dados de registro definido pelo usuário.

```
struct novo tipo
{
    tipo identificador;
    tipo identificador;
    ...
}
```

Abaixo está um exemplo de uso de **struct**. Neste exemplo a estrutura **aluno** é definida e são inicializadas duas instâncias, **a** e **b**, de duas maneiras diferentes. Finalmente, seu conteúdo é impresso.

```
struct aluno
{
```

```

char nome[50];
char matricula[14];
short int nascDia;
short int nascMes;
short int nascAno;
};

int main(void)
{
    struct aluno a = {
        .nome = "John Smith",
        .matricula = "202600001010",
        .nascDia = 31,
        .nascMes = 1,
        .nascAno = 2006,
    };
    struct aluno b = {"Johnson Smith", "202500002020", 1, 12,
        2005};

    printf("Nome:      %s\n", a.nome);
    printf("Matricula:  %s\n", a.matricula);
    printf("Nascimento: %02d/%02d/%d\n", a.nascDia, a.nascMes,
        a.nascAno);
    printf("\n");
    printf("Nome:      %s\n", b.nome);
    printf("Matricula:  %s\n", b.matricula);
    printf("Nascimento: %02d/%02d/%d\n", b.nascDia, b.nascMes,
        b.nascAno);
}

```

Na prática, geralmente se usa `typedef` para simplificar a declaração das variáveis. O `typedef` cria um alias para um tipo de dados definido pelo usuário, evitando a necessidade da palavra `struct` a cada declaração de variável desse tipo. A data também pode ser agregada em um outro tipo para simplificar seu uso caso necessário o uso em outras partes do código. Abaixo está a sintaxe usada.

```

typedef struct
{
    tipo identificador;
    tipo identificador;
    ...
} novo tipo;

```

Abaixo está um exemplo que mostra como o código anterior pode ser modi-

ficado para usar `typedef`, além de encapsular a data em um novo tipo.

```
typedef struct
{
    short int dia;
    short int mes;
    short int ano;
} data_t;

typedef struct
{
    char nome[50];
    char matricula[14];
    data_t nasc;
} aluno_t;

int main(void)
{
    aluno_t a = {
        .nome = "John Smith",
        .matricula = "202600001010",
        .nasc = {31, 1, 2006},
    };
    aluno_t b = {"Johnson Smith", "202500002020", 1, 12, 2005};

    printf("Nome:      %s\n", a.nome);
    printf("Matricula:  %s\n", a.matricula);
    printf("Nascimento: %02d/%02d/%d\n", a.nasc.dia, a.nasc.mes,
        a.nasc.ano);
    printf("\n");
    printf("Nome:      %s\n", b.nome);
    printf("Matricula:  %s\n", b.matricula);
    printf("Nascimento: %02d/%02d/%d\n", b.nasc.dia, b.nasc.mes,
        b.nasc.ano);
}
```

Tipos criados com `struct` podem ser passados como parâmetro, retornados e até usados em atribuições, como no exemplo abaixo. Experimente como exercício copiar, modificar e exibir o conteúdo da instância que recebeu a cópia para observar como ela realmente armazena uma nova cópia e não é simplesmente uma referência à instância original.

```
aluno_t alunoCopy(aluno_t src)
{
    aluno_t dst = src;
    return dst;
}
```

```

void alunoPrint(aluno_t a)
{
    printf("Nome:      %s\n", a.nome);
    printf("Matricula:  %s\n", a.matricula);
    printf("Nascimento: %02d/%02d/%d\n", a.nascDia, a.nascMes,
        a.nascAno);
}

```

Quando vários registros de mesmo tipo são necessários, podem ser armazenados como posições individuais em um vetor, como no exemplo abaixo. Observe que a inicialização agora não é mais feita exatamente como antes, sendo feita com um *type cast*.

```

aluno_t arrAluno[10];

arrAluno[0] = (aluno_t){
    .nome = "John Smith",
    .matricula = "202600001010",
    .nasc = {31, 1, 2006},
};
arrAluno[1] = (aluno_t){ "Johnson Smith", "202500002020", 1,
    12, 2005};

```

5.2 Estruturas compostas homogêneas

As estruturas compostas homogêneas são chamadas *arrays* e são usadas para representar vetores e matrizes. Vetores são representados por *arrays* unidimensionais e matrizes por *arrays* bidimensionais. O número de dimensões de um *array* é dado pelo número de índices usados para acessar uma posição. Também é possível se necessário ter mais dimensões. O número de posições possíveis de cada dimensão é seu tamanho.

Um vetor com três coordenadas é então representado por um *array* com uma dimensão de tamanho três. Uma matriz com três linhas e quatro colunas é representada por um *array* bidimensional com dimensões de tamanho três e quatro

Arrays são sequências de valores do mesmo tipo referenciados por um mesmo nome. Um elemento específico da sequência é acessado por meio de um índice inteiro não negativo. Na linguagem C, os *arrays* começam com o índice zero. Um *array* com *n* posições terá como índices válidos os inteiros de zero a *n*-1.

Uma declaração de *array* tem o seguinte formato:

Tipo de dado Identificador [Tamanho];

A expressão elementar abaixo funciona como uma variável comum, podendo ser usada em expressões aritméticas, de comparação, atribuição, ou como parâmetro em chamadas de funções.

Identificador [Posição]

Uma atribuição de valor a uma certa posição do *array*, por exemplo, tem o seguinte formato:

Identificador [Posição] = Valor;

Uma matriz é um *array* bidimensional. Sua declaração é semelhante à de um vetor unidimensional.

Tipo de dado Identificador [Linhas][Colunas];

Para acessar um item individual de uma matriz, usar o formato abaixo:

Identificador [Linha][Coluna]

5.3 Exercícios

1. Crie uma função **main**:

- (a) antes dela defina **TAM_MAX** como sendo 10 (dez).
- (b) dentro dela declare um vetor de inteiros chamado **vet**, com tamanho máximo igual **TAM_MAX**.
- (c) declare uma variável inteira chamada **tamanho**, para armazenar o tamanho ocupado desse vetor. Inicialize-a com zero.
- (d) crie um laço **for** que, a cada passo, solicita ao usuário que insira valores para cada posição do vetor, lê e armazena o dado digitado.
- (e) em caso de sucesso do **scanf**, incremente **tamanho** e continue. Em caso de erro, interrompa o laço com **break**.
- (f) depois do laço, imprima o tamanho ocupado do vetor. Note que o tamanho ocupado pode ser diferente do tamanho máximo **TAM_MAX**.
- (g) imprima o conteúdo de cada uma das posições do vetor usando a seguinte mensagem: **vet[%d] = %d\n** substituindo o primeiro **%d** pela posição, e o segundo pelo conteúdo.

2. Crie uma função `main` e dentro dela, declare um vetor de reais chamado `numeros`. Seu tamanho máximo é definido por uma macro chamada `TAM_MAX`, que vale 10 (dez). Em seguida, abra um arquivo chamado `input.txt` cujo conteúdo é um valor real em cada linha. Use `fscanf` para ler esse arquivo e armazenar os valores no vetor `numeros`, e o tamanho ocupado do vetor na variável inteira chamada `tamanho`. Seu programa deve funcionar para qualquer tamanho de entrada, inclusive para arquivos com mais de dez valores. Nesse caso, o vetor ficará completamente ocupado e o valor de `tamanho` será 10.
3. Crie um programa que aloca um *array* com tamanho 4×10 . Use o *array* para armazenar os valores abaixo considerando x variando de 1 a 10.
 - (a) Na primeira coluna o valor de x .
 - (b) Na segunda coluna o valor de x^2 .
 - (c) Na terceira coluna o valor de x^3 .
 - (d) Na quarta coluna o valor de x^4 .
 - (e) Imprima o conteúdo do *array*.
4. Crie um programa que testa se um certo n é primo. Armazene os primos encontrados em um vetor e use isso para acelerar o processamento. Use um laço para testar se os primos armazenados no vetor dividem n .
 - (a) Teste para $n = 101$.
 - (b) Solicite para o usuário que insira algum valor de n .
 - (c) Teste para n variando de 2 a 1000 e imprima o total de números primos encontrados.
 - (d) Teste para n variando de 3 a 9999 pulando os pares e imprima o total de números primos encontrados.
5. Arquivos com extensão PGM armazenam imagens do tipo *portable gray-map*. São arquivos muito simples, contendo como primeiro byte a letra P seguida de um dígito: 2 indica que as intensidades dos pixels estão em formato de texto e 5 indica formato binário. Linhas começando com # são comentários e devem ser ignoradas. Depois disso vêm três valores inteiros: largura, altura e intensidade máxima dos pixels, que tipicamente é 255 em imagens com 8 bits por amostra. Finalmente vêm os dados dos pixels, armazenados em formato *raster*, ou seja, primeiro a linha superior, da esquerda para a direita e assim por diante. Obtenha ou crie um arquivo PGM. Em seguida, crie uma função `main` dentro dela
 - (a) leia o arquivo e se certifique de que os primeiros bytes são compatíveis com o formato PGM.
 - (b) leia a largura e a altura da imagem e salve em duas variáveis.
 - (c) leia o valor da intensidade máxima dos pixels e se certifique de que é 255.

- (d) aloque um array de **char** com tamanho igual a altura $\times$ largura.
 - (e) leia os valores dos pixels e armazene no array alocado previamente.
6. Arquivos com extensão PPM armazenam imagens do tipo *portable pixmap*. São muito semelhantes aos arquivos PGM com algumas diferenças. Contêm como primeiro byte a letra P seguida de um dígito: 3 indica que as intensidades dos pixels estão em formato de texto e 6 indica formato binário. Linhas começando com # são comentários e devem ser ignoradas. Depois disso vêm três valores inteiros: largura, altura e intensidade máxima dos pixels, que tipicamente é 255 em imagens com 8 bits por amostra. Finalmente vêm os dados dos pixels, armazenados em formato *raster*. As amostras de cada pixel são armazenadas na ordem *red*, *green* e *blue*. Obtenha ou crie um arquivo PPM. Em seguida, crie uma função **maine** dentro dela
- (a) leia o arquivo e se certifique de que os primeiros bytes são compatíveis com o formato PPM.
 - (b) leia a largura e a altura da imagem e salve em duas variáveis.
 - (c) leia o valor da intensidade máxima dos pixels e se certifique de que é 255.
 - (d) aloque um array de **char** com tamanho igual a altura $\times$ largura $\times$ 3 para armazenar os três canais da imagem.
 - (e) leia os valores dos pixels e armazene no array alocado previamente.

Capítulo 6

Funções

Funções são blocos de instruções que executam uma certa tarefa. Podem receber dados de entrada e retornar dados de saída. Podemos dizer que a definição de uma função tem cinco partes: identificador ou nome, tipo de dado de retorno, lista de parâmetros, bloco de instruções e valor de retorno.

- Uma declaração de função tem a seguinte forma geral:

```
Tipo de dado Identificador (Lista de parâmetros)  
{  
    Bloco de código  
    return Valor de retorno;  
}
```

- Identificador é o nome através do qual chamamos a função. Alguns identificadores de funções já vistas são por exemplo `printf`, `scanf`, `fgets`, `sin` e `cos`.
- Tipo de dado de retorno é o tipo de dado que a função retorna. Como já foi visto na Seção 1.12, uma chamada a uma função é uma expressão elementar, e pode ser usada da mesma maneira que uma variável ou valor de mesmo tipo.

```
double x = sin(a);  
printf("%f\n", x);
```

Observe que `printf` imprimirá o valor de tipo `double` retornado pela função `sin`. De maneira alternativa, poderíamos colocar a chamada `sin` diretamente como parâmetro de `printf`.

```
printf("%f\n", sin(a));
```

- Lista de parâmetros são os dados de entrada da função. Cada parâmetro é uma variável cujo valor é definido na chamada da função. Cada item da lista tem o mesmo formato de uma declaração de variável, ou seja, é um tipo de dado seguido de um identificador. Se uma função recebe mais de um parâmetro, eles devem ser separados por vírgula.
- Bloco de código é o código executado quando a função é chamada.
- Valor de retorno é o valor retornado para a função que fez a chamada. Uma chamada a função pode fazer parte de uma expressão e tem o mesmo efeito que seu valor de retorno.

No exemplo abaixo podemos ver uma declaração bem simples de função. Essa função não recebe parâmetros e nem retorna valor. Simplesmente imprime uma mensagem e retorna. Logo após a declaração, vemos um exemplo de função `main` que a chama.

```
#include <stdio.h>
void cumprimenta(void)
{
    printf("Bom dia");
}

int main(void)
{
    cumprimenta();
}
```

Abaixo temos outro exemplo de função que não retorna valores. Essa função recebe um número de tipo `double` e o imprime usando notação científica.

```
#include <stdio.h>
void imprimed(double val)
{
    printf("%.10E", val);
}

int main(void)
{
    imprimed(299792458.0);
    printf("\n");
}
```

Vale notar que o tipo de dado de retorno da função não tem relação com os tipos dos parâmetros. Podem ser completamente diferentes como no exemplo abaixo, que recebe `float` como entrada e retorna `int`. A expressão `(int)` executa um *typecast*, ou seja, uma conversão de tipo.

```
#include <stdio.h>
int arredonda(float val)
{
    return (int) (val+0.5);
}

int main(void)
{
    printf("Pi eh aproximadamente %d\n", arredonda(3.14159265));
}
```

6.1 Passagem de parâmetros por valor

Quando chamamos uma função, os valores dos parâmetros são copiados para as variáveis definidas na lista de parâmetros. Podemos modificar esses valores no interior da função sem que isso tenha efeito fora dela.

```
#include <stdio.h>
int dobra(int n)
{
    n = 2 * n;
    return n;
}

int main(void)
{
    int a = 5;
    int b = dobra(a);
    printf("O dobro de %d eh %d\n", a, b);
}
```

O valor de `a` é copiado para o parâmetro `n` da função. `n` recebe 10 mas `a` permanece 5.

6.2 Passagem de parâmetros por referência

Esse tipo de passagem de parâmetro é necessário quando a função deve ser capaz de alterar alguma variável fora de seu escopo. Em outras palavras, quando precisamos que a função altere alguma variável da função que a chamou. Um exemplo de passagem de parâmetro por referência é o que fazemos ao chamar `scanf`.

Nesse tipo de passagem de parâmetro, ao invés de passarmos um valor, passamos um ponteiro, ou seja, uma referência à posição de memória que gostaríamos de alterar.

No exemplo abaixo, a função **troca** é implementada. Ela troca os valores de duas variáveis da função que a chamou. Isso só é possível usando passagem de parâmetro por referência.

```
#include <stdio.h>
int troca(int *a, int *b)
{
    int aux = *a;
    *a = *b;
    *b = aux;
}

int main(void)
{
    int a = 5;
    int b = 3;
    printf("Antes: x = %d, y = %d\n", x, y);
    troca(&x, &y);
    printf("Depois: x = %d, y = %d\n", x, y);
}
```

6.3 Exercícios

1. Crie uma função chamada **alo** que não recebe parâmetros nem retorna valores, apenas imprime a mensagem "hello world". Resposta:

```
void alo(void)
{
    printf("hello world\n");
}
```

2. Crie uma função **main** que simplesmente chama a função **alo**.
3. Crie uma função de tipo **void** chamada **preguica**, que não recebe parâmetros e não faz nada.
4. Crie uma função **main** que simplesmente chama a função **preguica**.
5. Crie uma função chamada **imprimeInt** que recebe um parâmetro inteiro, mas não retorna valores, apenas imprime a mensagem "valor inteiro igual a " seguida do valor do parâmetro recebido.

6. Crie uma função `main` que simplesmente chama a função `imprimeInt`. Lembre-se de fornecer como parâmetro algum valor inteiro.
7. Crie uma função chamada `duzia` que recebe um parâmetro real de precisão simples e retorna 12 vezes seu valor. Resposta:

```
float duzia(float x)
{
    return 12*x;
}
```

8. Crie uma função `main` que simplesmente chama a função `duzia`. Lembre-se de fornecer como parâmetro algum valor de tipo `float`. Salve o resultado em uma variável e imprima o resultado.
9. Crie uma função de tipo `int` chamada `triplo`, que recebe um parâmetro inteiro e retorna três vezes seu valor.
10. Crie uma função `main` que chama a função `triplo`, salva o valor retornado em uma variável e em seguida o imprime.
11. Crie uma função de tipo `int` chamada `eh_par`, que recebe um parâmetro inteiro e retorna 1 (um) caso o inteiro seja par, e 0 (zero) caso contrário.
12. Crie uma função de tipo `void` chamada `imprime_float`, que recebe um parâmetro de tipo `float` e imprime a mensagem:
"o valor da variavel real eh %f\n"
substituindo o `%f` pelo valor do parâmetro.
13. Crie uma função de tipo `float` chamada `circunferencia`, que recebe um parâmetro de tipo `float` igual ao raio de um círculo e retorna o comprimento de sua circunferência.
14. Crie uma função de tipo `float` chamada `area_circulo`, que recebe um parâmetro de tipo `float` igual ao raio de um círculo e retorna seu raio.
15. Crie uma função de tipo `float` chamada `volume_esfera`, que recebe um parâmetro de tipo `float` igual ao raio de uma esfera e retorna seu volume.
16. Crie uma função de tipo `int` chamada `somatoria`, que recebe um parâmetro de tipo `int` chamado `n` e retorna a soma dos inteiros de 1 a `n`.
17. Crie uma função chamada `fatorial` que recebe um parâmetro inteiro e retorna seu fatorial.
 - a) implemente a função usando um laço `while`.
 - b) implemente a função usando um laço `for`.

c) implemente a função usando uma chamada a ela mesma. Para isso, use a definição recursiva de fatorial:

se `n == 0` (`==` é operador de comparação)

então `fatorial(n) = 1`

senão `fatorial(n) = n * fatorial(n-1)`

18. Crie uma função de tipo `int` chamada `somatoria`, que recebe um parâmetro de tipo `int` chamado `n` e retorna a soma dos inteiros de 1 a `n`.
19. Crie uma função de tipo `double` chamada `norma`, que recebe dois parâmetros de tipo `double` chamados `a` e `b`, e retorna a norma do vetor (a, b) .
20. Crie uma função chamada `pg3`, que retorna o terceiro termo de uma progressão geométrica. A função recebe dois parâmetros inteiros, o primeiro parâmetro é o primeiro termo da pg; o segundo é sua razão.
21. Crie uma função chamada `pg`, que retorna o `n`-ésimo termo de uma progressão geométrica. A função recebe três parâmetros inteiros, o primeiro parâmetro é o primeiro termo da pg; o segundo é sua razão, o terceiro é o `n`.
22. Crie uma função de tipo `void` chamada `swap` que recebe dois parâmetros inteiros por referência e troca seus valores.
 - (a) Crie uma função `main` e dentro dela declare e inicialize duas variáveis inteiras. Imprima seus valores, chame a função `swap` passando-as por referência e imprima novamente, para testar se os valores foram realmente trocados.
23. Crie uma função de tipo `double` chamada `delta`, que recebe três parâmetros de tipo `double` chamados `a`, `b` e `c`, e retorna o valor de delta, ou seja, $b^2b - 4.0*a*c$.
24. Crie uma função de tipo `int` chamada `baskara`. Ela recebe por valor três parâmetros de tipo `int` chamados `a`, `b` e `c`, representando os coeficientes de uma equação de segundo grau igual a $a*x*x + b*x + c$. Recebe ainda dois parâmetros por referência, chamados `x1` e `x2`, onde armazenará as raízes encontradas. Ela retorna o número de raízes reais da equação, ou seja, pode retornar 0 (zero), 1 (um) ou 2 (dois). Usar a função delta da questão anterior para testar quantas raízes a equação possui.

Capítulo 7

Exemplos de programas

7.1 pause.c

```
/* Programa que imprime uma mensagem e termina quando o
   usuario pressiona enter.
*/
#include <stdio.h>

int main(void)
{
    printf("Pressione enter para continuar...");
    getchar();
}
```

Listagem 7.1: series.c

7.2 ascii.c

```
/* Programa que imprime os 128 caracteres da tabela ASCII.
*/
# include <stdio.h>

#define NUMCOL 4
#define MAXCHAR 127

main()
{
    const char* arrName[] = {"NUL", "STX", "SOT", "ETX", "EOT",
                             "ENQ", "ACK", "BEL", "BS", "HT", "LF", "VT", "FF", "CR",
```

```

"SO", "SI", "DLE", "DC1", "DC2", "DC3", "DC4", "NAK", "SYN",
"ETB", "CAN", "EM", "SUB", "ESC", "FS", "GS", "RS", "US"};
const int arrNameSize = 32;

for(int i = 0; i < (MAXCHAR + 1) / NUMCOL; i++)
{
    for(int j = 0; j < NUMCOL; j++)
    {
        int c = j * ((MAXCHAR + 1) / NUMCOL) + i;
        if (c < arrNameSize)
        {
            printf("| %3d 0x%02X %3s ", c, c, arrName[c]);
        }
        else
        {
            printf("| %3d 0x%02X %3c ", c, c, c);
        }
    }
    printf("\n");
}
}

```

Listagem 7.2: ascii.c

7.3 dado.c

```

/* Programa que lança um dado com tantos lados quanto
definidos em LADOS e imprime o valor sorteado.
Cada sorteio resulta em um valor maior
ou igual a 1 e menor ou igual a LADOS.
O numero de sorteios eh definido em SORTEIOS.
O programa conta quantas vezes cada
lado foi sorteado e imprime a contagem no final.
*/

#include <stdio.h> // printf, getchar
#include <stdlib.h> // rand, srand
#include <time.h> // time

#define LADOS 6
#define SORTEIOS 10000

int main(void)
{

```



```

    srand(time(NULL));
    int count[LADOS + 1];

    for (int i = 0; i <= LADOS; i++)
    {
        count[i] = 0;
    }

    for (int i = 0; i < SORTEIOS; i++)
    {
        int result = rand() % LADOS + 1;
        printf("valor = %d\n", result % LADOS + 1);
        //getchar();
        count[result]++;
    }

    for (int i = 0; i <= LADOS; i++)
    {
        printf("count[%d] = %d\n", i, count[i]);
    }
}

```

Listagem 7.3: dado.c

7.4 primos.c

```

/* Este programa solicita ao usuario que digite um
numero inteiro. O numero eh lido e armazenado em
uma variavel. Em seguida o programa calcula seus
fatores primos e os imprime.
*/

#include <stdio.h>

int main()
{
    int val;
    printf("digite um inteiro:\n");
    int result = scanf("%d", &val);
    printf("result = %d, val = %d\n", result, val);

    int quociente = val;
    for (int i = 2; i <= val; i++)
    {
        int resto;

```

```

do
{
    resto = quociente % i;
    //printf("resto = %d, quoc = %d, i = %d\n", resto,
quociente, i);
    if (resto == 0)
    {
        printf("fator: %d\n", i);
        quociente = quociente / i;
    }
}
while (resto == 0);
}
}

```

Listagem 7.4: series.c

7.5 series.c

```

/* Este programa solicita ao usuario que digite um
numero inteiro. O numero eh lido e armazenado em
uma variavel. Em seguida o programa calcula seu
fatorial, seu namero de fibonacci e imprime o
resultado.
*/

#include <stdio.h>
#include <cstdlib>

int fibonacci(int val)
{
    if (val == 0)
    {
        return 0;
    }
    else if (val == 1)
    {
        return 1;
    }
    else
    {
        return fibonacci(val - 1) + fibonacci(val - 2);
    }
}

int fatorial(int val)

```

```

{
    if (val <= 1)
    {
        return 1;
    }
    else
    {
        return val * fatorial(val - 1);
    }
}

int main()
{
    int val;
    printf("digite um inteiro:\n");

    int coderr = scanf("%d", &val);
    printf("coderr = %d, val = %d\n", coderr, val);

    int result;

    result = fatorial(val);
    printf("fatorial(%d) = %d\n", val, result);

    result = fibonacci(val);
    printf("fibonacci(%d) = %d\n", val, result);

    system("pause");
}

```

Listagem 7.5: series.c

7.6 generate.c

```

/* Gera uma lista de numeros aleatorios entre 0 e MAXNUM
   inclusive. Passar como parametro um inteiro igual a
   quantidade de numeros a serem gerados.
   Para criar um arquivo com os numeros gerados, suceder
   a chamada ao programa com > e o nome do arquivo.
   Exemplo:
   generate 1000 > saida.txt
*/

#include <stdlib.h>
#include <stdio.h>

```

```

#define DEBUG 0
#define MAXNUM 100000

int main (int argc, char** argv)
{
    if (DEBUG){
        printf("argc = %d\n", argc);
        for (int i = 0; i < argc; i++)
        {
            printf("argv[%d] = %s\n", i, argv[i]);
        }
    }

    if (argc < 2)
    {
        fprintf(stderr, "Error: Missing required parameter
        <n>.\n\n");
        fprintf(stderr, "Usage: %s <n>\n", argv[0]);
        fprintf(stderr, "  <n> : An integer value (e.g., 42)\n");
        exit(1);
    }

    int size;
    int result = sscanf(argv[1], "%d", &size);

    if (result != 1)
    {
        exit(1);
    }

    for (int i = 0; i < size; i++)
    {
        int number = rand() % MAXNUM + 1;
        printf("%d\n", number);
    }
}

```

Listagem 7.6: generate.c

7.7 ordena.c

```

/* Este programa recebe como parametro um nome
   de arquivo com um inteiro em cada linha.
   Le o arquivo, armazena os inteiros em um
   vetor e os ordena, imprimindo o tempo

```

```

    gasto na ordenacao.
*/

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define MAXTAM 1000000

int comparacoes = 0;
int trocas       = 0;

#define swap(arr, a, b)      \
{                             \
    int __swap_buf__ = arr[a]; \
    arr[a] = arr[b];          \
    arr[b] = __swap_buf__;    \
}

int readArrayFromFile(char* filename, int* arr, int* size)
{
    FILE* fp = fopen(filename, "r");

    if (fp == 0)
    {
        return 1;
    }

    int result;
    int i = 0;
    do
    {
        result = fscanf(fp, "%d", &arr[i]);
        i++;
    }
    while (result == 1);
    *size = i - 1;

    return 0;
}

void printArray(int* arr, int size)
{
    for (int i = 0; i < size; i++)
    {
        printf("arr[%7d] = %7d\n", i, arr[i]);
    }
}

```

```

    }
}

void copyArray(int* src, int* dst, int size)
{
    for (int i = 0; i < size; i++)
    {
        dst[i] = src[i];
    }
}

void bubbleSort(int* arr, int size)
{
    comparacoes = 0;
    trocas = 0;
    for (int maxi = size - 2; maxi >= 1; maxi--)
    {
        for (int i = 0; i <= maxi; i++)
        {
            comparacoes++;
            if (arr[i] > arr[i+1])
            {
                trocas++;
                swap(arr, i, i+1);
            }
        }
    }
}

void selectionSort(int* arr, int size)
{
    for (int maxi = size - 1; maxi >= 1; maxi--)
    {
        int imaxval = 0;
        for (int i = 1; i <= maxi; i++)
        {
            comparacoes++;
            if (arr[i] > arr[imaxval])
            {
                imaxval = i;
            }
        }
        comparacoes++;
        if (imaxval != maxi)
        {
            trocas++;

```

```

        swap(arr, maxi, imaxval);
    }
}

void mergeSortMerge(int* arr, int i0, int j0, int i1)
{
    int size = i1 - i0 + 1;
    int* aux = (int*) malloc(size * sizeof (int));
    int i = i0;
    int j = j0;
    for (int k = 0; k < size; k++)
    {
        comparacoes += 3;
        if ( (i < j0) && ( (j > i1) || (arr[i] < arr[j]) ) )
        {
            aux[k] = arr[i];
            i++;
        }
        else
        {
            aux[k] = arr[j];
            j++;
        }

        /*
        if (j > i1)
        {
            aux[k] = arr[i];
            i++;
        }

        else if (i == j0){
            aux[k] = arr[j];
            j++;
        }

        else if(arr[i] < arr[j])
        {
            aux[k] = arr[i];
            i++;
        }

        else
        {
            aux[k] = arr[j];
            j++;
        }
        */
    }
}

```

```

    }
    for (int k = 0; k < size; k++)
    {
        trocas++;
        arr[i0 + k] = aux[k];
    }
    free(aux);
}

void mergeSortRec(int* arr, int i0, int i1)
{
    comparacoes++;
    if(i0 < i1)
    {
        mergeSortRec(arr, i0, (i0+i1)/2);
        mergeSortRec(arr, (i0+i1)/2 + 1, i1);
        mergeSortMerge(arr, i0, (i0+i1)/2 + 1, i1);
    }
}

void mergeSort(int* arr, int size)
{
    comparacoes = 0;
    trocas = 0;
    mergeSortRec(arr, 0, size);
}

double timer(int start)
{
    static struct timespec t0;
    static struct timespec t1;

    if (start)
    {
        clock_gettime(CLOCK_REALTIME, &t0);
        //printf("t0 sec = %ld\n", t0.tv_sec);
        //printf("t0 nsec = %ld\n", t0.tv_nsec);
        return 0.0;
    }
    else
    {
        clock_gettime(CLOCK_REALTIME, &t1);
        //printf("t1 sec = %ld\n", t1.tv_sec);
        //printf("t1 nsec = %ld\n", t1.tv_nsec);

        double delta = (t1.tv_sec - t0.tv_sec) +

```



```

        ( (t1.tv_nsec - t0.tv_nsec) /
1000000000.0 );

        return delta;
    }
}

void timerStart(void)
{
    timer(1);
}

double timerEnd(void)
{
    return timer(0);
}

int main(int argc, char** argv)
{
    if (argc < 2)
    {
        exit(1);
    }

    int arr0[MAXTAM];
    int arr[MAXTAM];

    int size;
    int error = readArrayFromFile(argv[1], arr0, &size);

    if (error)
    {
        exit(1);
    }

    printf("size = %d\n", size);
    //printArray(arr, size);

    printf("CLOCKS_PER_SEC = %ld\n", CLOCKS_PER_SEC);

    double delta;

    copyArray(arr0, arr, size);
    timerStart();
    selectionSort(arr, size);

```

```

    delta = timerEnd();
    printf("Bubble sort:\n");
    printf("Comparacoes: %5d\n", comparacoes);
    printf("Trocas:      %5d\n", trocas);
    printf("Time:        %f sec\n", delta);

    copyArray(arr0, arr, size);
    timerStart();
    selectionSort(arr, size);
    delta = timerEnd();
    printf("Selection sort:\n");
    printf("Comparacoes: %5d\n", comparacoes);
    printf("Trocas:      %5d\n", trocas);
    printf("Time:        %f sec\n", delta);

    copyArray(arr0, arr, size);
    timerStart();
    mergeSort(arr, size);
    delta = timerEnd();
    printf("Merge sort:\n");
    printf("Comparacoes: %5d\n", comparacoes);
    printf("Trocas:      %5d\n", trocas);
    printf("Time:        %f sec\n", delta);

    //printArray(arr, size);
}

```

Listagem 7.7: ordena.c

Apêndice A

Tabela ASCII

Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char
0	0x00	NUL	32	0x20		64	0x40	@	96	0x60	'
1	0x01	STX	33	0x21	!	65	0x41	A	97	0x61	a
2	0x02	SOT	34	0x22	"	66	0x42	B	98	0x62	b
3	0x03	ETX	35	0x23	#	67	0x43	C	99	0x63	c
4	0x04	EOT	36	0x24	\$	68	0x44	D	100	0x64	d
5	0x05	ENQ	37	0x25	%	69	0x45	E	101	0x65	e
6	0x06	ACK	38	0x26	&	70	0x46	F	102	0x66	f
7	0x07	BEL	39	0x27	'	71	0x47	G	103	0x67	g
8	0x08	BS	40	0x28	(	72	0x48	H	104	0x68	h
9	0x09	HT	41	0x29	)	73	0x49	I	105	0x69	i
10	0x0A	LF	42	0x2A	*	74	0x4A	J	106	0x6A	j
11	0x0B	VT	43	0x2B	+	75	0x4B	K	107	0x6B	k
12	0x0C	FF	44	0x2C	,	76	0x4C	L	108	0x6C	l
13	0x0D	CR	45	0x2D	-	77	0x4D	M	109	0x6D	m
14	0x0E	SO	46	0x2E	.	78	0x4E	N	110	0x6E	n
15	0x0F	SI	47	0x2F	/	79	0x4F	O	111	0x6F	o
16	0x10	DLE	48	0x30	0	80	0x50	P	112	0x70	p
17	0x11	DC1	49	0x31	1	81	0x51	Q	113	0x71	q
18	0x12	DC2	50	0x32	2	82	0x52	R	114	0x72	r
19	0x13	DC3	51	0x33	3	83	0x53	S	115	0x73	s
20	0x14	DC4	52	0x34	4	84	0x54	T	116	0x74	t
21	0x15	NAK	53	0x35	5	85	0x55	U	117	0x75	u
22	0x16	SYN	54	0x36	6	86	0x56	V	118	0x76	v
23	0x17	ETB	55	0x37	7	87	0x57	W	119	0x77	w
24	0x18	CAN	56	0x38	8	88	0x58	X	120	0x78	x
25	0x19	EM	57	0x39	9	89	0x59	Y	121	0x79	y
26	0x1A	SUB	58	0x3A	:	90	0x5A	Z	122	0x7A	z
27	0x1B	ESC	59	0x3B	;	91	0x5B	[	123	0x7B	{
28	0x1C	FS	60	0x3C	<	92	0x5C		124	0x7C	
29	0x1D	GS	61	0x3D	=	93	0x5D	]	125	0x7D	}
30	0x1E	RS	62	0x3E	>	94	0x5E	^	126	0x7E	
31	0x1F	US	63	0x3F	?	95	0x5F	_	127	0x7F	DEL